

Assignment No -8

Name - Payal Balasaheb More . **Roll No** - EN23107080 . **Class** - TY(B) . **Batch** - A

Title - Implement AI security measures to protect against adversarial attacks and data breaches.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from cryptography.fernet import Fernet
```

```
df = pd.read_csv("cybersecurity_attacks.csv")
```

```
df.describe()
```

	Source Port	Destination Port	Packet Length	Anomaly Scores
count	40000.000000	40000.000000	40000.000000	40000.000000
mean	32970.356450	33150.868650	781.452725	50.113473
std	18560.425604	18574.668842	416.044192	28.853598
min	1027.000000	1024.000000	64.000000	0.000000
25%	16850.750000	17094.750000	420.000000	25.150000
50%	32856.000000	33004.500000	782.000000	50.345000
75%	48928.250000	49287.000000	1143.000000	75.030000
max	65530.000000	65535.000000	1500.000000	100.000000

```
df.shape
```

```
(40000, 25)
```

```
df.isnull().sum()
```

	0
Timestamp	0
Source IP Address	0
Destination IP Address	0
Source Port	0
Destination Port	0
Protocol	0
Packet Length	0
Packet Type	0
Traffic Type	0
Payload Data	0
Malware Indicators	20000
Anomaly Scores	0
Alerts/Warnings	20067
Attack Type	0
Attack Signature	0
Action Taken	0
Severity Level	0
User Information	0
Device Information	0
Network Segment	0
Geo-location Data	0
Proxy Information	19851
Firewall Logs	19961
IDS/IPS Alerts	20050
Log Source	0

dtype: int64

```
df = df.drop(columns=[
    "Timestamp",
    "Source IP Address",
    "Destination IP Address",
    "Payload Data",
    "Malware Indicators"
])
```

df

	Source Port	Destination Port	Protocol	Packet Length	Packet Type	Traffic Type	Anomaly Scores	Alerts/Warnings	Attack Type	Attack Signature	Action Taken	Severity Level	In	
0	31225	17616	ICMP	503	Data	HTTP	28.67		NaN	Malware	Known Pattern B	Logged	Low	
1	17245	48166	ICMP	1174	Data	HTTP	51.50		NaN	Malware	Known Pattern A	Blocked	Low	S
2	16811	53600	UDP	306	Control	HTTP	87.42	Alert Triggered	DDoS	Known Pattern B	Ignored	Low		
3	20018	32534	UDP	385	Data	HTTP	15.79	Alert Triggered	Malware	Known Pattern B	Blocked	Medium		
4	6131	26646	TCP	1462	Data	DNS	0.52	Alert Triggered	DDoS	Known Pattern B	Blocked	Low		
...	...	...	...	...	...	...	...	...	...	...	...	...		
39995	31005	6764	UDP	1428	Control	HTTP	39.28		NaN	DDoS	Known Pattern A	Logged	Medium	A
39996	2553	28091	UDP	1184	Control	HTTP	27.25		NaN	DDoS	Known Pattern A	Logged	High	
39997	22505	25152	UDP	1043	Data	DNS	31.01		NaN	DDoS	Known Pattern B	Blocked	Low	Sa
39998	20013	2703	UDP	483	Data	FTP	97.85	Alert Triggered	Malware	Known Pattern B	Ignored	Low		
39999	50137	55575	ICMP	1175	Control	HTTP	34.63	Alert Triggered	Intrusion	Known Pattern B	Logged	Medium	Z	

40000 rows × 20 columns

```
df.shape
(40000, 20)
```

```
encoder = LabelEncoder()

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = encoder.fit_transform(df[col].astype(str))
```

```
X = df.drop("Attack Type", axis=1)
y = df["Attack Type"]
```

```
importances = model.feature_importances_
print(importances)

[0.10389987 0.10322299 0.02467547 0.10033057 0.01461966 0.02484285
 0.10294387 0.01393288 0.01481908 0.02419668 0.02434131 0.10336132
 0.105235 0.02390772 0.10354729 0.06848469 0.01423601 0.01505678
 0.01434594]
```

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from sklearn.model_selection import GridSearchCV

params = {
    "n_estimators": [200, 300],
    "max_depth": [10, 15, 20]
}

grid = GridSearchCV(RandomForestClassifier(), params, cv=5)

grid.fit(X_train, y_train)

print(grid.best_params_)

{'max_depth': 10, 'n_estimators': 200}
```

```
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    random_state=42
)

model.fit(X_train, y_train)
```

RandomForestClassifier

RandomForestClassifier(max_depth=15, n_estimators=200, random_state=42)

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

Accuracy: 0.336375

Classification Report:

	precision	recall	f1-score	support
0	0.34	0.37	0.35	2636
1	0.35	0.29	0.32	2721
2	0.33	0.35	0.34	2643
accuracy			0.34	8000
macro avg	0.34	0.34	0.34	8000
weighted avg	0.34	0.34	0.34	8000

Confusion Matrix:

[[670 715 643]